

Winsorization Sensitivity Analysis (Diagnostic)

Purpose. RMSE and MAE for all 6 models are large in absolute RM terms. This notebook checks whether that is because the models are poorly trained, or because a small proportion of very expensive listings (long right tail of `price`) disproportionately inflate the error metrics - especially RMSE, since it squares errors before averaging (Section 1.8.1).

Method. For a given percentile p , a price threshold is computed from `X_train`'s price distribution only. Prices above that threshold - in both `X_train` and `X_test` - are winsorized (clipped) down to the threshold, and `y_train / y_test` (`log(price)`) are recomputed from the clipped prices. Fitting the threshold on `X_train` only and re-using it on `X_test` mirrors the train-only-fit principle already used throughout this project. Each model is retrained with its **already-tuned hyperparameters** (loaded from the saved `.pkl`) - no re-tuning.

This notebook is diagnostic only - it does not change `data_preprocessing.ipynb`, any `model_*.ipynb`, or any saved `.pkl`.

```
In [ ]: import os
import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
import joblib
from sklearn.base import clone

sys.path.append(os.path.join(os.path.dirname(os.path.abspath('__file__')), '..', 'model'))
from model_utils import evaluate_model

MODELLING_DIR = os.path.join(".", "model", "modelling_data")
MODEL_DIR = os.path.join(".", "streamlit")

X_train_raw = pd.read_csv(os.path.join(MODELLING_DIR, "X_train.csv"))
X_test_raw = pd.read_csv(os.path.join(MODELLING_DIR, "X_test.csv"))
X_train_scaled = pd.read_csv(os.path.join(MODELLING_DIR, "X_train_scaled.csv"))
X_test_scaled = pd.read_csv(os.path.join(MODELLING_DIR, "X_test_scaled.csv"))
y_train_orig = pd.read_csv(os.path.join(MODELLING_DIR, "y_train.csv"))["price"]
y_test_orig = pd.read_csv(os.path.join(MODELLING_DIR, "y_test.csv"))["price"]

price_train = np.exp(y_train_orig)
price_test = np.exp(y_test_orig)

rm_formatter = FuncFormatter(lambda x, _: f"RM {x:,.0f}")

print(f"X_train: {X_train_raw.shape} | X_test: {X_test_raw.shape}")
print(f"Price range (train): RM {price_train.min():,.0f} - RM {price_train.max():,.0f}")
print(f"Price median (train): RM {price_train.median():,.0f}")
```

X_train: (3004, 51) | X_test: (751, 51)
Price range (train): RM 38,000 - RM 6,016,000
Price median (train): RM 350,000

1. Winsorization Cutoff Price by Percentile

The price cutoff (in RM) at each candidate percentile, computed from `X_train` only.

```
In [ ]: percentiles = [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]
scan_rows = []
for p in percentiles:
    val = price_train.quantile(p / 100)
    n_above = int((price_train > val).sum())
    scan_rows.append({"percentile": p, "threshold_RM": val, "n_rows_above": n_above,
                     "pct_of_train": n_above / len(price_train) * 100})
scan_df = pd.DataFrame(scan_rows)
print(scan_df.to_string(index=False, formatters={"threshold_RM": "{:,.0f}".format, "pct_of_train": "{:.2f}%".format}))
```

percentile	threshold_RM	n_rows_above	pct_of_train
90	688,000	300	9.99%
91	708,000	270	8.99%
92	747,600	241	8.02%
93	780,000	206	6.86%
94	822,975	181	6.03%
95	889,700	151	5.03%
96	958,800	121	4.03%
97	1,100,000	88	2.93%
98	1,297,480	61	2.03%
99	1,599,400	31	1.03%

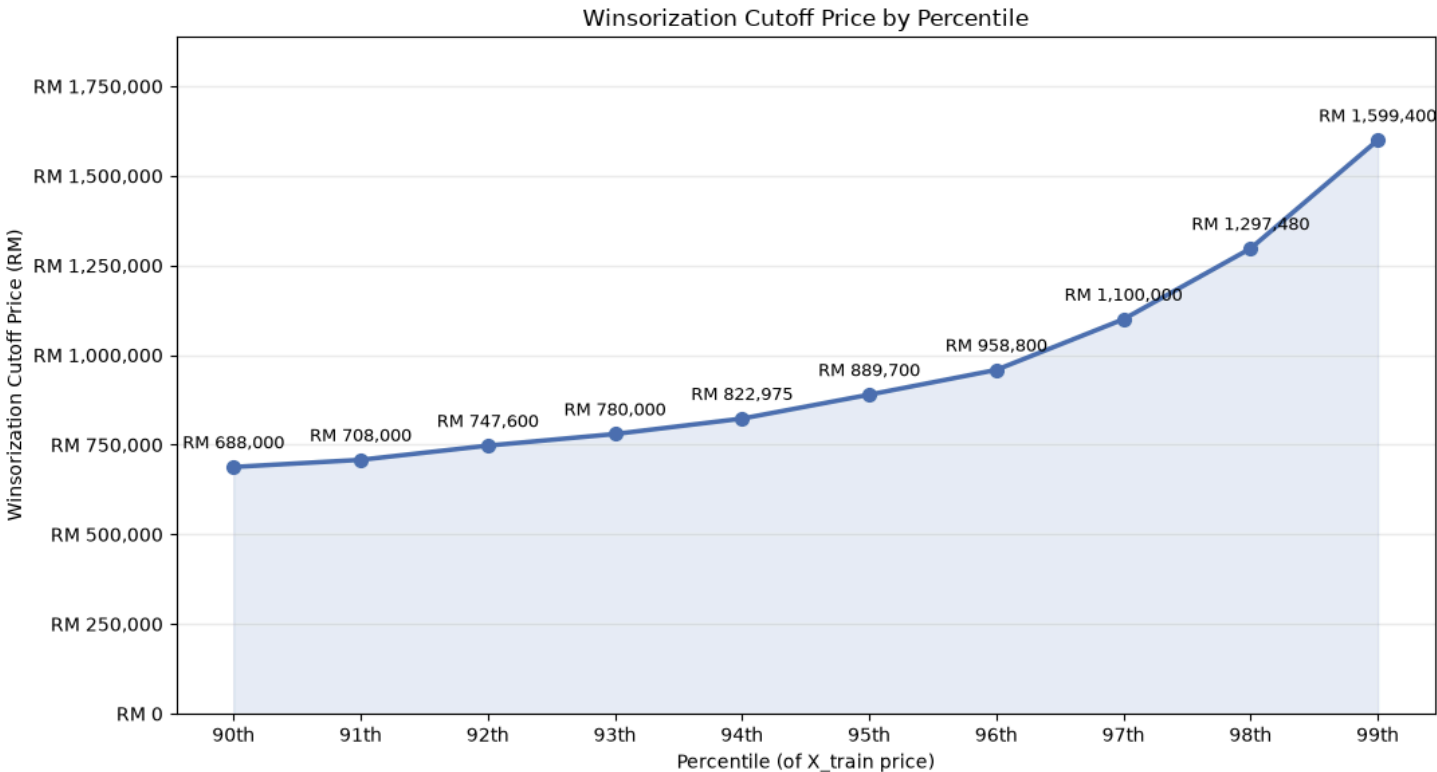
```
In [ ]: fig, ax = plt.subplots(figsize=(11, 6))
labels = [f"{p}th" for p in percentiles]
x_pos = range(len(percentiles))

ax.plot(x_pos, scan_df["threshold_RM"], marker="o", markersize=7, linewidth=2.5,
        color="#4c72b0", zorder=3)
ax.fill_between(x_pos, scan_df["threshold_RM"], color="#4c72b0", alpha=0.12)

ax.yaxis.set_major_formatter(rm_formatter)
for x, val in zip(x_pos, scan_df["threshold_RM"]):
    ax.annotate(f"RM {val:,.0f}", (x, val), textcoords="offset points", xytext=(0, 10),
                ha="center", fontsize=9)

ax.set_xticks(list(x_pos))
ax.set_xticklabels(labels)
ax.set_xlabel("Percentile (of X_train price)")
ax.set_ylabel("Winsorization Cutoff Price (RM)")
```

```
ax.set_title("Winsorization Cutoff Price by Percentile")
ax.set_ylim(0, scan_df["threshold_RM"].max() * 1.18)
ax.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.show()
```



2. Winsorize Example: 97th Percentile

Threshold fit on `X_train` price only, then the same cutoff applied to `X_test` . Each model is retrained (existing tuned hyperparameters, reused via `clone()`) on the winsorized target, and compared against the uncapped baseline already reported in Section 5.2/6.1.

```
In [ ]: CAP_PERCENTILE = 0.97
threshold_97 = price_train.quantile(CAP_PERCENTILE)

price_train_capped = price_train.clip(upper=threshold_97)
price_test_capped = price_test.clip(upper=threshold_97)
n_train_capped = int((price_train > threshold_97).sum())
n_test_capped = int((price_test > threshold_97).sum())

y_train_capped = np.log(price_train_capped)
y_test_capped = np.log(price_test_capped)

print(f"Winsorize threshold (97th pct of X_train price): RM {threshold_97:,.0f}")
print(f"X_train rows capped: {n_train_capped} / {len(price_train)} ({n_train_capped/len(price_train)*100:.2f}%)")
print(f"X_test rows capped: {n_test_capped} / {len(price_test)} ({n_test_capped/len(price_test)*100:.2f}%)")
print(f"New price range - train: RM {price_train_capped.min():,.0f} to RM {price_train_capped.max():,.0f}")
print(f"New price range - test: RM {price_test_capped.min():,.0f} to RM {price_test_capped.max():,.0f}")

Winsorize threshold (97th pct of X_train price): RM 1,100,000
X_train rows capped: 88 / 3004 (2.93%)
X_test rows capped: 26 / 751 (3.46%)
New price range - train: RM 38,000 to RM 1,100,000
New price range - test: RM 69,999 to RM 1,100,000
```

```
In [ ]: BASELINES = {
    "Linear Regression": {"test_rmse": 194955, "test_mae": 92474},
    "Ridge Regression": {"test_rmse": 194955, "test_mae": 92474},
    "Random Forest": {"test_rmse": 175825, "test_mae": 76856},
    "XGBoost": {"test_rmse": 170218, "test_mae": 80785},
    "SVR": {"test_rmse": 195720, "test_mae": 88335},
    "KNN": {"test_rmse": 200777, "test_mae": 102096},
}

MODEL_FILES = {
    "Linear Regression": ("linear_model.pkl", "scaled"),
    "Ridge Regression": ("ridge_model.pkl", "scaled"),
    "Random Forest": ("random_forest_model.pkl", "raw"),
    "XGBoost": ("xgboost_model.pkl", "raw"),
    "SVR": ("svr_model.pkl", "scaled"),
    "KNN": ("knn_model.pkl", "scaled"),
}

fitted_models = {}
for name, (fname, data_type) in MODEL_FILES.items():
    path = os.path.join(MODEL_DIR, fname)
    fitted_models[name] = (joblib.load(path), data_type)
print(f"Loaded {len(fitted_models)} models: {list(fitted_models.keys())}")

Loaded 6 models: ['Linear Regression', 'Ridge Regression', 'Random Forest', 'XGBoost', 'SVR', 'KNN']
```

```
In [ ]: results_97 = {}

for name, (fitted, data_type) in fitted_models.items():
    fresh = clone(fitted)
    X_train = X_train_raw if data_type == "raw" else X_train_scaled
    X_test = X_test_raw if data_type == "raw" else X_test_scaled
```

```
fresh.fit(X_train, y_train_capped)
test_m = evaluate_model(fresh, X_test, y_test_capped)

base = BASELINES[name]
results_97[name] = {
    "Test RMSE (baseline)": base["test_rmse"], "Test RMSE (97th capped)": test_m["RMSE"],
    "Test MAE (baseline)": base["test_mae"], "Test MAE (97th capped)": test_m["MAE"],
}

results_97_df = pd.DataFrame(results_97).T.round(2)
results_97_df
```

c:\Users\Low\anaconda3\Lib\site-packages\sklearn\svm_base.py:340: ConvergenceWarning: Solver terminated early (max_iter=5000). Consider pre-processing your data with StandardScaler or MinMaxScaler.

warnings.warn(

	Test RMSE (baseline)	Test RMSE (97th capped)	Test MAE (baseline)	Test MAE (97th capped)
Linear Regression	194955.0	117919.96	92474.0	76988.94
Ridge Regression	194955.0	117919.85	92474.0	76988.86
Random Forest	175825.0	98038.38	76856.0	61792.60
XGBoost	170218.0	95447.87	80785.0	63962.87
SVR	195720.0	109872.82	88335.0	70519.97
KNN	200777.0	130527.05	102096.0	86580.79

3. Test RMSE vs Winsorization Cutoff

Repeats the winsorize experiment at 90th, 93rd, 95th, 97th, and 99th percentiles to show how Test RMSE changes as more of the price tail is capped.

```
In [ ]: PERCENTILES = [90, 93, 95, 97, 99]
sweep_results = {name: [] for name in fitted_models}
sweep_thresholds = []

for pct in PERCENTILES:
    threshold = price_train.quantile(pct / 100)
    sweep_thresholds.append(threshold)
    price_train_c = price_train.clip(upper=threshold)
    price_test_c = price_test.clip(upper=threshold)
    y_train_c = np.log(price_train_c)
    y_test_c = np.log(price_test_c)

    for name, (fitted, data_type) in fitted_models.items():
        fresh = clone(fitted)
        X_train = X_train_raw if data_type == "raw" else X_train_scaled
        X_test = X_test_raw if data_type == "raw" else X_test_scaled
        fresh.fit(X_train, y_train_c)
        test_m = evaluate_model(fresh, X_test, y_test_c)
        sweep_results[name].append(test_m["RMSE"])
    print(f"{pct}th percentile (RM {threshold:,.0f}) done")

print("\nSweep complete.")
```

c:\Users\Low\anaconda3\Lib\site-packages\sklearn\svm_base.py:340: ConvergenceWarning: Solver terminated early (max_iter=5000). Consider pre-processing your data with StandardScaler or MinMaxScaler.

warnings.warn(

90th percentile (RM 688,000) done

c:\Users\Low\anaconda3\Lib\site-packages\sklearn\svm_base.py:340: ConvergenceWarning: Solver terminated early (max_iter=5000). Consider pre-processing your data with StandardScaler or MinMaxScaler.

warnings.warn(

93th percentile (RM 780,000) done

c:\Users\Low\anaconda3\Lib\site-packages\sklearn\svm_base.py:340: ConvergenceWarning: Solver terminated early (max_iter=5000). Consider pre-processing your data with StandardScaler or MinMaxScaler.

warnings.warn(

95th percentile (RM 889,700) done

c:\Users\Low\anaconda3\Lib\site-packages\sklearn\svm_base.py:340: ConvergenceWarning: Solver terminated early (max_iter=5000). Consider pre-processing your data with StandardScaler or MinMaxScaler.

warnings.warn(

97th percentile (RM 1,100,000) done

c:\Users\Low\anaconda3\Lib\site-packages\sklearn\svm_base.py:340: ConvergenceWarning: Solver terminated early (max_iter=5000). Consider pre-processing your data with StandardScaler or MinMaxScaler.

warnings.warn(

99th percentile (RM 1,599,400) done

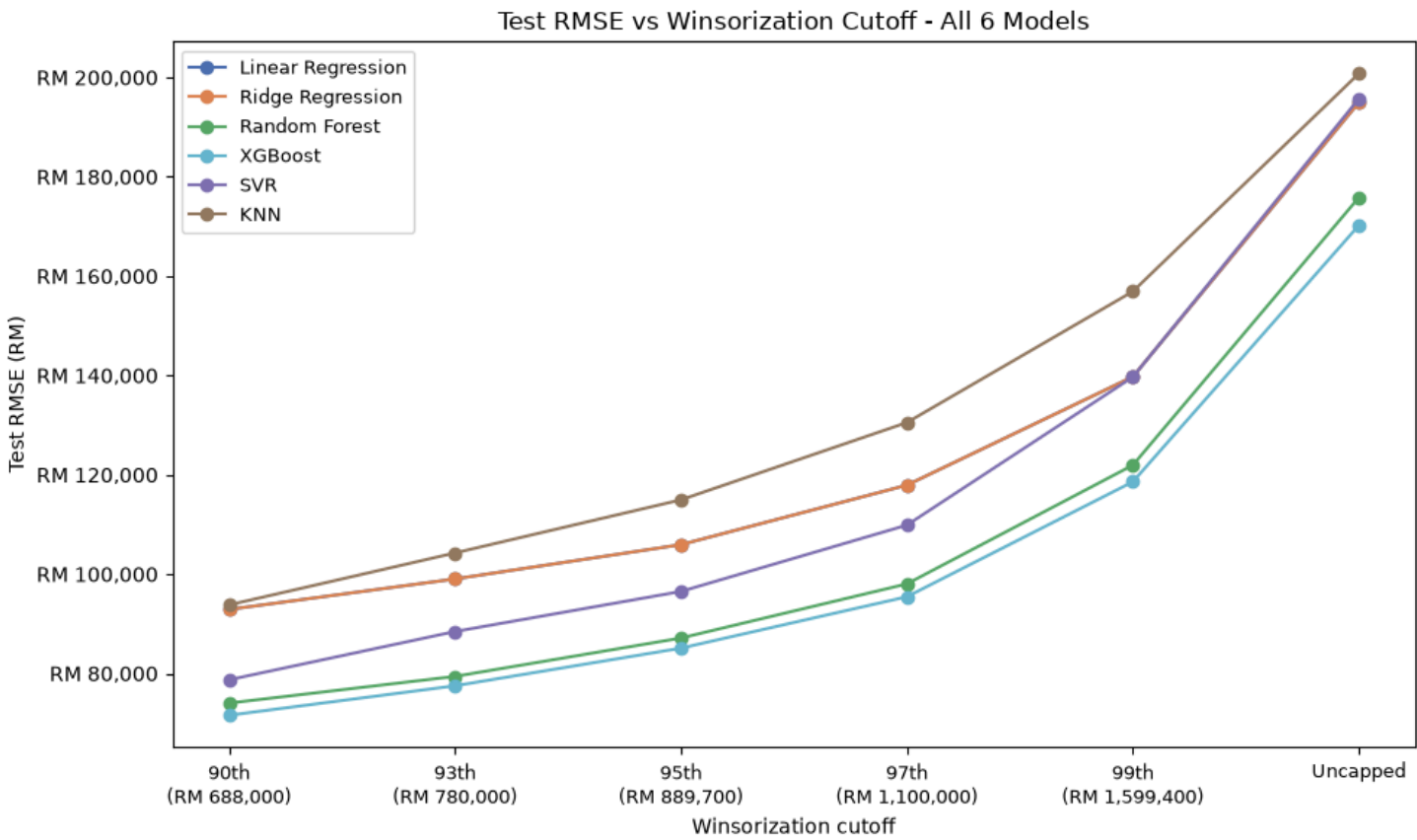
Sweep complete.

```
In [ ]: fig, ax = plt.subplots(figsize=(10, 6))
x_labels = [f"{p}th\n(RM {t:,.0f})" for p, t in zip(PERCENTILES, sweep_thresholds)] + ["Uncapped"]
line_colors = ["#4c72b0", "#dd8452", "#55a868", "#64b5cd", "#8172b2", "#937860"]

for (name, values), color in zip(sweep_results.items(), line_colors):
    full_values = values + [BASELINES[name]["test_rmse"]]
    ax.plot(range(len(x_labels)), full_values, marker="o", label=name, color=color)

ax.yaxis.set_major_formatter(rm_formatter)
ax.set_xticks(range(len(x_labels)))
ax.set_xticklabels(x_labels, fontsize=9)
ax.set_xlabel("Winsorization cutoff")
ax.set_ylabel("Test RMSE (RM)")
ax.set_title("Test RMSE vs Winsorization Cutoff - All 6 Models")
ax.legend(fontsize=9)
```

```
plt.tight_layout()
plt.show()
```



```
In [ ]: sweep_table = pd.DataFrame(sweep_results, index=[f"{p}th pct" for p in PERCENTILES])
sweep_table.loc["Uncapped"] = [BASELINES[name]["test_rmse"] for name in sweep_results]
sweep_table
```

	Linear Regression	Ridge Regression	Random Forest	XGBoost	SVR	KNN
90th pct	92896.859264	92896.671937	74039.377263	71588.061967	78727.920617	93805.667302
93th pct	99036.720665	99036.527579	79412.579541	77520.578977	88446.130227	104269.870108
95th pct	105938.824555	105938.646269	87133.466948	85097.856731	96518.108460	114964.625231
97th pct	117919.961287	117919.849170	98038.376960	95447.870306	109872.819769	130527.051272
99th pct	139749.183168	139749.301209	121949.887416	118600.797475	139694.144059	156920.281749
Uncapped	194955.000000	194955.000000	175825.000000	170218.000000	195720.000000	200777.000000